

Name: Student ID:

ECS401: Unit 9 Lab Progress Test

These exercises are to help you judge your progress. It is really important that you make sure you fully understand everything we have covered so far.

Dry running recursive calls: Overview

When dry running method calls each call can be given its own dry run table. For recursive calls (methods that call themselves) each new table has the same columns, just with different values as in this example.

When you get to the recursive call you have to pause this table and start to fill out the next one as shown by the solid arrows overleaf. Only when the result of the call is returned can you fill in the rest of the table (here the blue columns).

The return value from the next table is used as the result of evaluating the recursive call as shown by the dotted arrows overleaf.

```
public static int fact(int n)    // L1
{
    if (n == 1)                  // L2
        return 1;                // L3
    else
    {
        System.out.print(n);      // L4
        return (n * fact(n-1));    // L5
    }
}
```

Assuming the call:

```
fact(4);
```

The dry run table overleaf shows that the return value is 24 as that is the value eventually returned by the original call of `fact(4)` as shown in the first table.

We have added arrows to show how calls link the tables and how values are used once returned.

What is printed is:

```
432
```

as each value is printed BEFORE the recursive call. If the values were printed after the recursive call returned then the later table print statements would execute before the earlier ones so the printing would happen in the opposite order.

fact(4)					
LINE	n	n == 1 ?	print	fact(n-1)	return (n * fact(n-1))
L1	4				
L2		4 == 1 ? false			
L4			4		
L5				fact(3) =? 6	24

fact(3)					
LINE	n	n == 1 ?	print	fact(n-1)	return (n * fact(n-1))
L1	3				
L2		4 == 1 ? false			
L4			3		
L5				fact(2) =? 2	6

fact(2)					
LINE	n	n == 1 ?	print	fact(n-1)	return (n * fact(n-1))
L1	2				
L2		4 == 1 ? false			
L4			2		
L5				fact(1) =? 1	2

fact(1)					
LINE	n	n == 1 ?	print	fact(n-1)	return 1
L1	1				
L2		1 == 1 ? true			
L3					1

1. Complete the dry run tables and give both the value ultimately returned and the text that is printed to the screen after a call `f(2)`; when `f` is defined as below.

```

public static int f(int k)    // L1
{
    if (k == 5)               // L2
    {
        System.out.print(k); // L3
        return 10;           // L4
    }
    else
    {
        System.out.print(k); // L5
        return (k + f(k+1)); // L6
    }
}

```

f()					
LINE	k	k == 5 ?	print	f(k+1)	return (k + f(k+1))

f()					
LINE	k	k == 5 ?	print	f(k+1)	return (k + f(k+1))

f()					
LINE	k	k == 5 ?	print	f(k+1)	return (k + f(k+1))

f()					
LINE	k	k == 5 ?	print	f(k+1)	return 10

Printed:

Returned:

2. Complete the dry run table and give the text that is printed to the screen after a call. `printme(4)` ; when `printme` is defined as below:

```
public static void printme(int n) // L1
{
    if (n == 0) // L2
    {
        System.out.print(1); // L3
        return; // L4
    }
    else
    {
        System.out.print(n+1); // L5
        return; // L6
    }
}
```

Fill out the dry run table below to work out the answer. HINT: this method is NOT recursive.

printme()				
LINE	n	n == 0 ?	print	return

Printed:

3. Complete the dry run table and give the text that is printed to the screen after a call. `printme(4)` ; when `printme` is defined as below:

```
public static void printme(int n) // L1
{
    if (n == 0) // L2
    {
        System.out.print(1); // L3
        return; // L4
    }
    else
    {
        System.out.print(n+1); // L5
        printme(n-2); // L6
        return; // L7
    }
}
```

printme()					
LINE	n	n == 0 ?	print	printme(n-2)	return

printme()					
LINE	n	n == 0 ?	print	printme(n-2)	return

printme()					
LINE	n	n == 0 ?	print	printme(n-2)	return

Printed:

4. Complete the dry run table and give the text that is printed to the screen after a call. `printme(4)` ; when `printme` is defined as below:

```

public static void printme(int n) // L1
{
    if (n == 0) // L2
    {
        System.out.print(1); // L3
        return; // L4
    }
    else
    {
        printme(n-2); // L5
        System.out.print(n+1); // L6
        return; // L7
    }
}

```

printme()					
LINE	n	n == 0 ?	printme(n-2)	print	return

printme()					
LINE	n	n == 0 ?	printme(n-2)	print	return

printme()					
LINE	n	n == 0 ?	printme(n-2)	print	return

Printed:

5. In a recursive function the idea is that on each step you leave a smaller, easier version of the problem in the sense that you move closer to the base case. In the method in question 4 above what is:

a) The step case:

b) The base case: